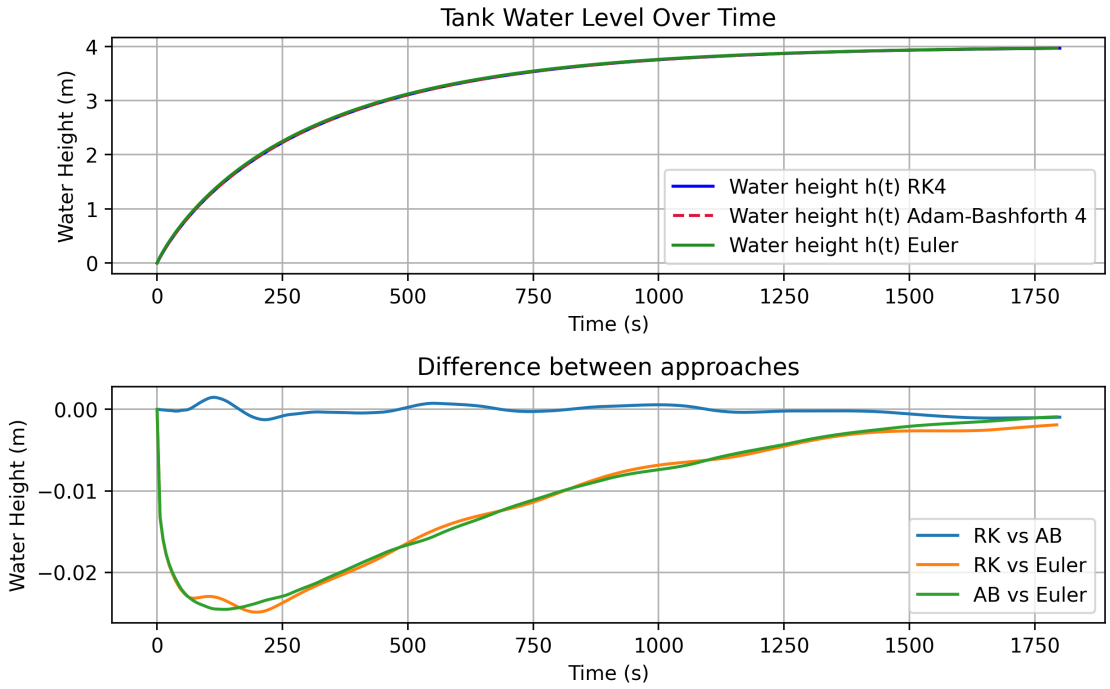
ODE

April 27, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp # function that i use to solve the␣
 ↪initial value problem.

# Our known parameters needed
A = 100           # Cross-sectional area
Q_in = 2.0       # constant flow into system
C = 1.0          # Constant of C from our formula

# Time span
t_span = (0, 1800)    # 30 minutes or 1800 seconds
step = 6              # intervals, so this is every 6 seconds.
t_eval = np.arange(0,t_span[-1]+step,step)

h0 = [0.0]
h_step = 0.1

# here i build a function based on our formula: dh/dt = (Q_in - C * sqrt(h)) / A
def tank_ode(t, h):
    outflow = C * np.sqrt(max(h, 0))
    dhdt = (Q_in - outflow) / A
    return dhdt

def euler_method(h0, t_span, h, ode_func):

    '''Recall the equation we describe is y_n+1 = y_n +hf'(xn)'''
    t_values = np.arange(t_span[0], t_span[1], h)
    h_values = [h0]

    for i in range(1, len(t_values)):
        t = t_values[i-1]
        h_current = h_values[-1]
        dhdt = ode_func(t, h_current)  # rate of change of height
        h_next = h_current + h * dhdt
        h_values.append(h_next)
```

```
    return t_values, np.array(h_values)

euler_t, euler_h = euler_method(0.0, t_span, np.nanmean(np.diff(t_eval)),␣
 ↪tank_ode)


'''Thanks to scipy, we do not necessarily have to manually code the other two␣
 ↪approaches!!'''

# Solve the ODE
sol1 = solve_ivp(tank_ode, t_span, h0, t_eval=t_eval,method='RK45')
"""Built in function for RK45:
https://github.com/scipy/scipy/blob/main/scipy/integrate/_ivp/rk.py
"""

sol2 = solve_ivp(tank_ode, t_span, h0, t_eval=t_eval,method='LSODA')
"""Built in function for A-B:
https://github.com/scipy/scipy/blob/main/scipy/integrate/_ivp/lsoda.py
"""

plt.figure(figsize=(8, 5),dpi=320)
plt.subplot(2,1,1)
plt.plot(sol1.t, np.ravel(sol1.y), label='Water height h(t) RK4', color='blue')
plt.plot(sol2.t, np.ravel(sol2.y), '--',label='Water height h(t) Adam-Bashforth␣
 ↪4', color='crimson')
plt.plot(euler_t, euler_h, label='Water height h(t) Euler',␣
 ↪color='forestgreen', linestyle='-')
plt.xlabel('Time (s)')
plt.ylabel('Water Height (m)')
plt.title('Tank Water Level Over Time')
plt.grid(True)
plt.legend()

plt.subplot(2,1,2)
plt.plot(sol1.t, np.ravel(sol1.y)- np.ravel(sol2.y),label='RK vs AB')
plt.plot(sol1.t[:-1], np.ravel(sol1.y)[:-1]- np.ravel(euler_h),label='RK vs␣
 ↪Euler')
plt.plot(sol1.t[:-1], np.ravel(sol2.y)[:-1]- np.ravel(euler_h),label='AB vs␣
 ↪Euler')

plt.legend()
plt.xlabel('Time (s)')
plt.ylabel('Water Height (m)')
plt.title('Difference between approaches')
plt.grid(True)
plt.tight_layout()
```

```
plt.show()
```



1 Lets change some things

How does the height look after 1800 seconds?

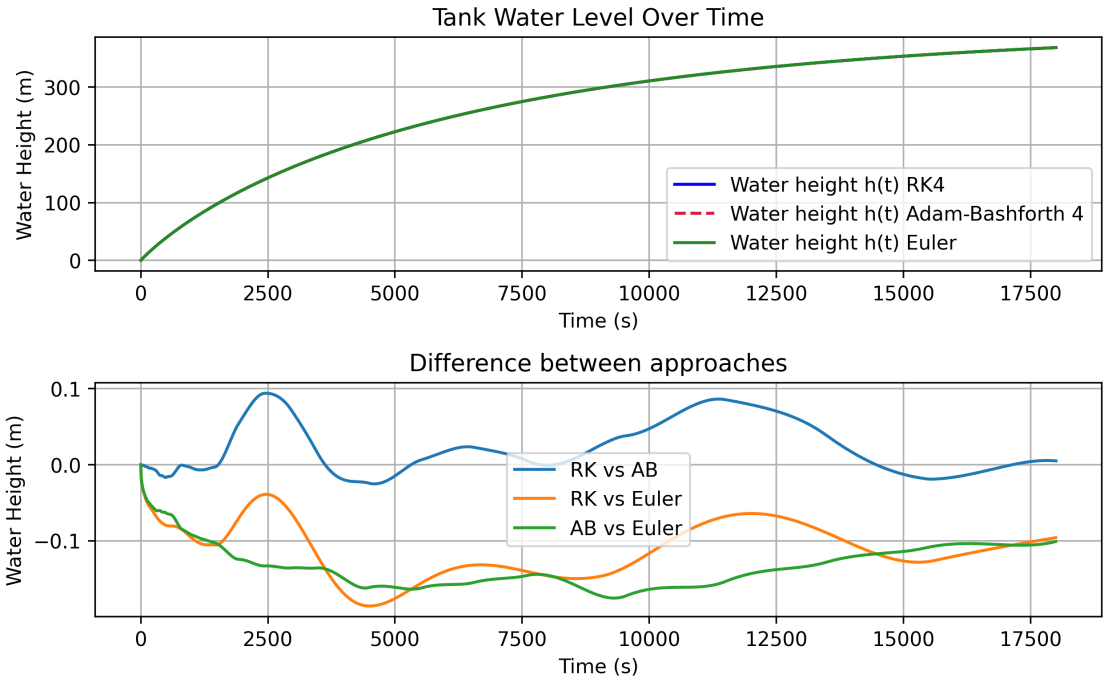
What happens if we look at a CSA of 250m2?

Doubling __ ?

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp # function that i use to solve the␣
 ↪initial value problem.

# Our known parameters needed
A = 200           # Cross-sectional area
Q_in = 20.0       # constant flow into system
C = 1.0          # Constant of C from our formula

# Time span
t_span = (0, 18000)   # 30 minutes or 1800 seconds
step = 6
```

```
t_eval = np.arange(0,t_span[-1]+step,step) # intervals, so this is every 6␣
 ↪seconds.

h0 = [0.0]
h_step = 0.1

# here i build a function based on our formula: dh/dt = (Q_in - C * sqrt(h)) / A
def tank_ode(t, h):
    outflow = C * np.sqrt(max(h, 0))
    dhdt = (Q_in - outflow) / A
    return dhdt

def euler_method(h0, t_span, h, ode_func):

    '''Recall the equation we describe is y_n+1 = y_n +hf'(xn)'''
    t_values = np.arange(t_span[0], t_span[1], h)
    h_values = [h0]

    for i in range(1, len(t_values)):
        t = t_values[i-1]
        h_current = h_values[-1]
        dhdt = ode_func(t, h_current)  # rate of change of height
        h_next = h_current + h * dhdt
        h_values.append(h_next)

    return t_values, np.array(h_values)

euler_t, euler_h = euler_method(0.0, t_span, np.nanmean(np.diff(t_eval)),␣
 ↪tank_ode)


'''Thanks to scipy, we do not necessarily have to manually code the other two␣
 ↪approaches!!'''

# Solve the ODE
sol1 = solve_ivp(tank_ode, t_span, h0, t_eval=t_eval,method='RK45')
"""Built in function for RK45:
https://github.com/scipy/scipy/blob/main/scipy/integrate/_ivp/rk.py
"""

sol2 = solve_ivp(tank_ode, t_span, h0, t_eval=t_eval,method='LSODA')
"""Built in function for A-B:
https://github.com/scipy/scipy/blob/main/scipy/integrate/_ivp/lsoda.py
"""

plt.figure(figsize=(8, 5),dpi=320)
plt.subplot(2,1,1)
```

```
plt.plot(sol1.t, np.ravel(sol1.y), label='Water height h(t) RK4', color='blue')
plt.plot(sol2.t, np.ravel(sol2.y), '--',label='Water height h(t) Adam-Bashforth␣
 ↪4', color='crimson')
plt.plot(euler_t, euler_h, label='Water height h(t) Euler',␣
 ↪color='forestgreen', linestyle='-')
plt.xlabel('Time (s)')
plt.ylabel('Water Height (m)')
plt.title('Tank Water Level Over Time')
plt.grid(True)
plt.legend()

plt.subplot(2,1,2)
plt.plot(sol1.t, np.ravel(sol1.y)- np.ravel(sol2.y),label='RK vs AB')
plt.plot(sol1.t[:-1], np.ravel(sol1.y)[:-1]- np.ravel(euler_h),label='RK vs␣
 ↪Euler')
plt.plot(sol1.t[:-1], np.ravel(sol2.y)[:-1]- np.ravel(euler_h),label='AB vs␣
 ↪Euler')

plt.legend()
plt.xlabel('Time (s)')
plt.ylabel('Water Height (m)')
plt.title('Difference between approaches')
plt.grid(True)
plt.tight_layout()
plt.show()
```



[3]:
```
print("what about a random flow in: *np.random.rand()")
```

```
what about a random flow in: *np.random.rand()
```

[ ]:
```

```